

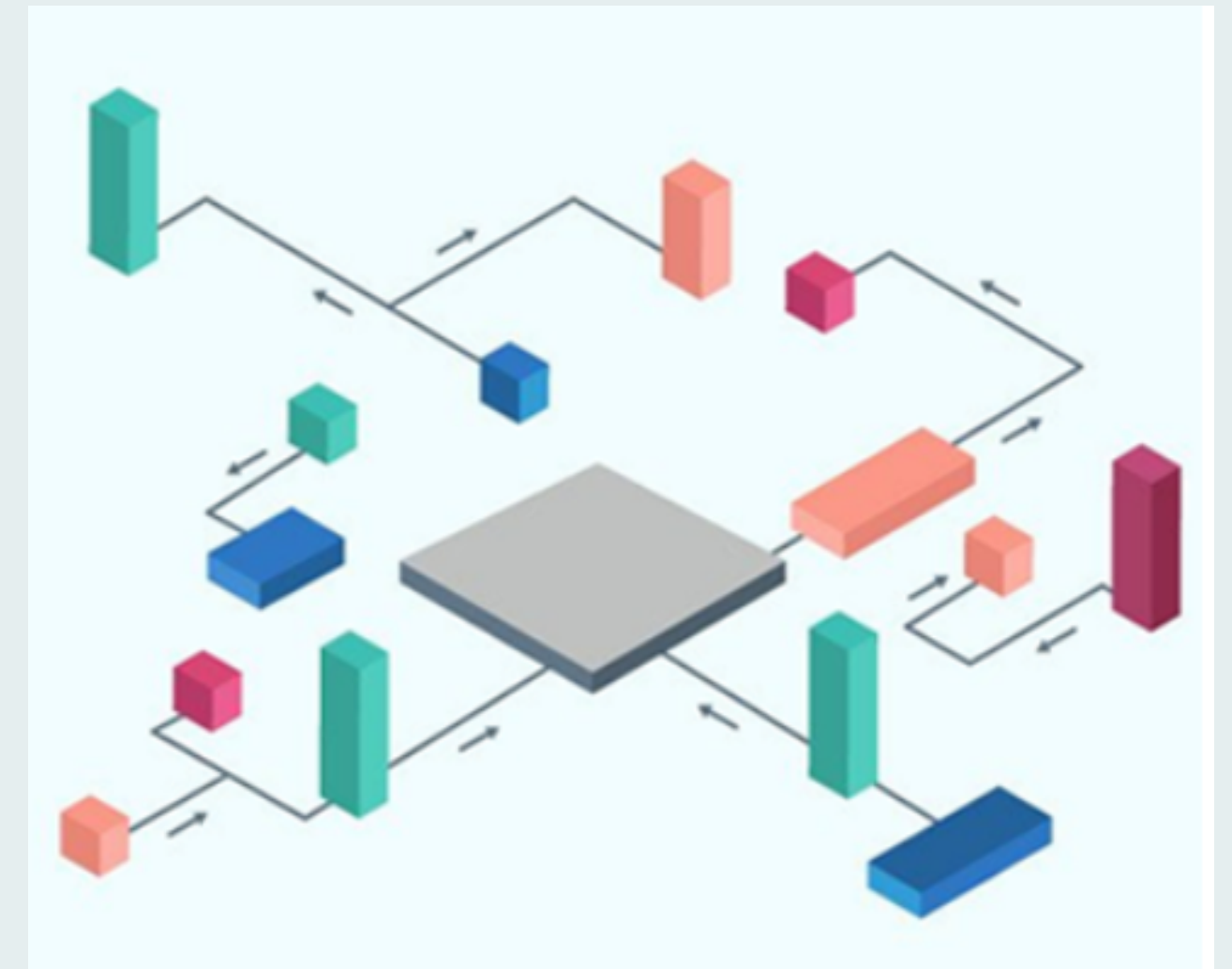
Microservices & Design Patterns



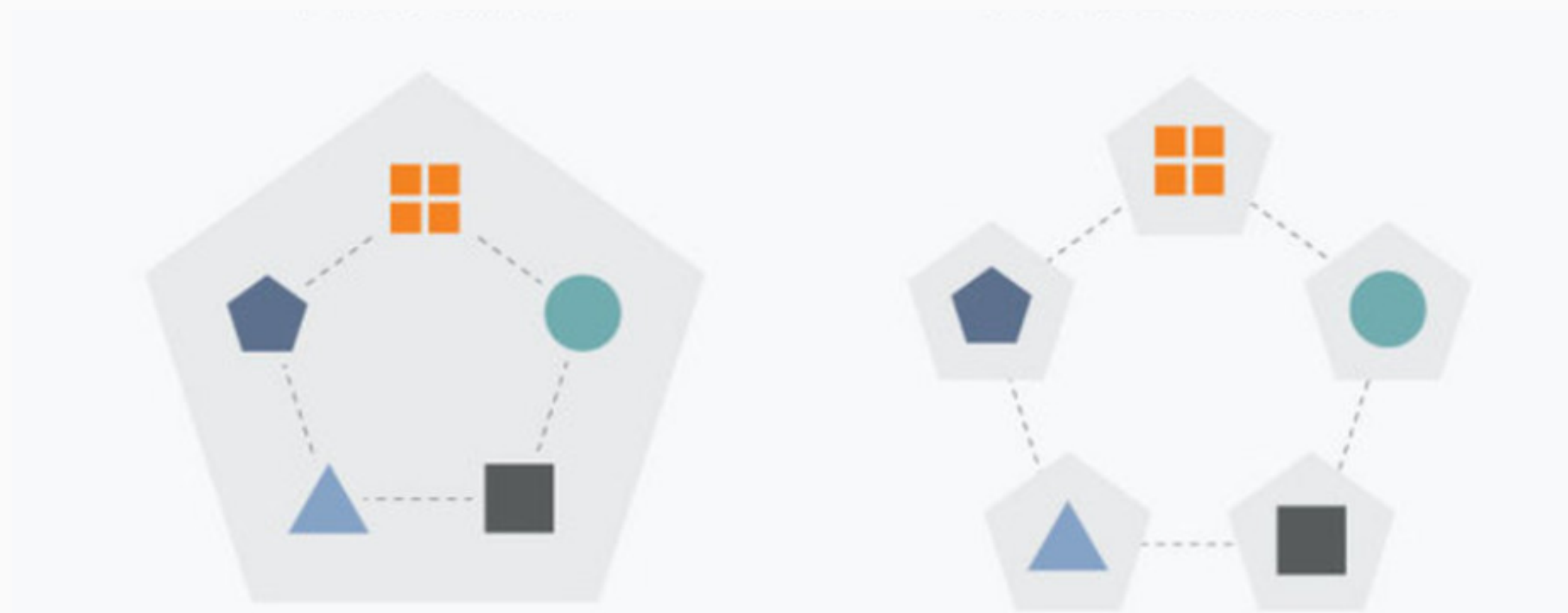
What will we discuss?



- What are Microservices?
- Why are they helpful?
- Useful libraries to ease life with Microservices.
- What design patterns to use to make Microservice experience more exciting?



What is a microservice architecture?



Microservices are an architectural approach to building applications. Microservices are distributed and loosely coupled. In well designed architecture changes in one microservice have minimal impact on the entire application.



Pros

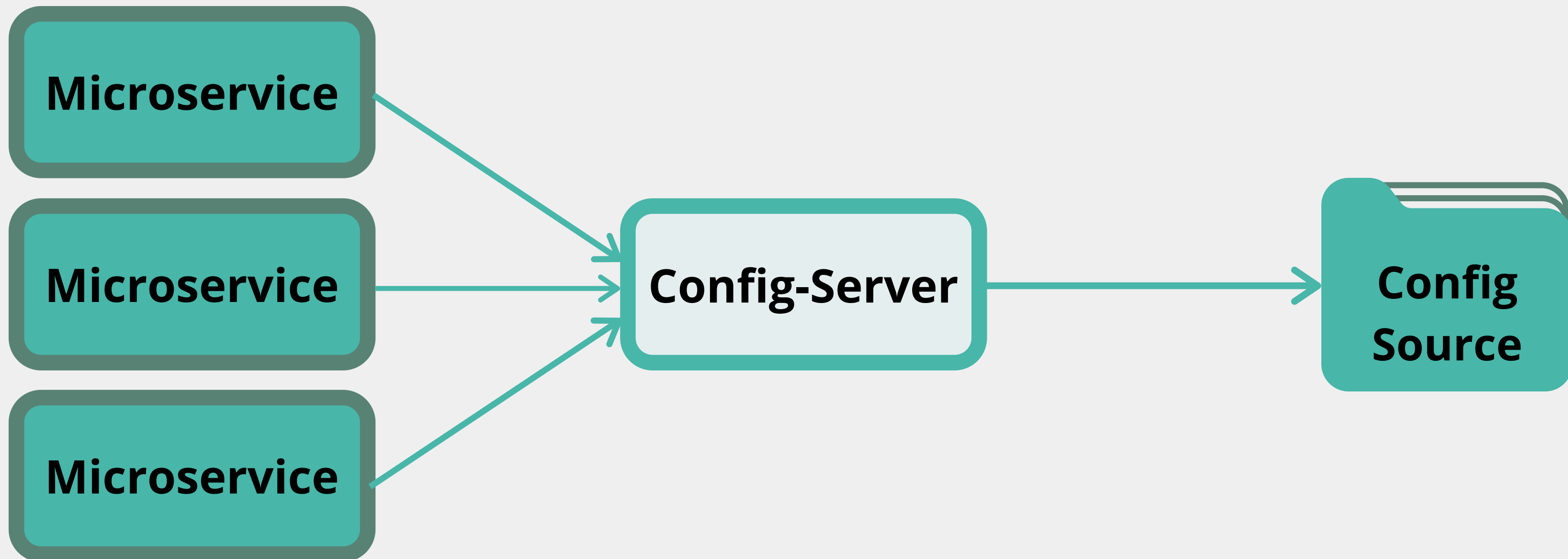
- Flexible -new functionality is delivered easily and quickly
- Resilient - Failures can be localized to a small part of application and corrected before the entire application experiences an outage
- Scalable - scaling of a system with small services is localized and much more cost-effective
- Platform independence

Cons

- Communication between services is complex
- More services equals more resources. Multiple databases and transaction management can be painful
- Complex testing over a distributed environment

Spring Cloud Config

Spring Cloud Config provides server and client-side support for externalized configuration in a distributed system. With the Config Server you have a central place to manage external properties for applications across all environments.



Config-Server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServer {

    public static void main(String[] arguments) {
        SpringApplication.run(ConfigServer.class, arguments);
    }
}
```

```
server.port=8888
spring.cloud.config.server.git.uri=ssh://localhost/config-repo
spring.cloud.config.server.git.clone-on-start=true
spring.security.user.name=root
spring.security.user.password=s3cr3t
```

Config Server Client

We need to have bootstrap.properties file

```
spring.application.name=config-client  
spring.profiles.active=development  
spring.cloud.config.uri=http://localhost:8888  
spring.cloud.config.username=root  
spring.cloud.config.password=s3cr3t
```

```
<dependency>  
    <groupId>org.springframework.cloud</groupId>  
    <artifactId>spring-cloud-config-client</artifactId>  
</dependency>
```

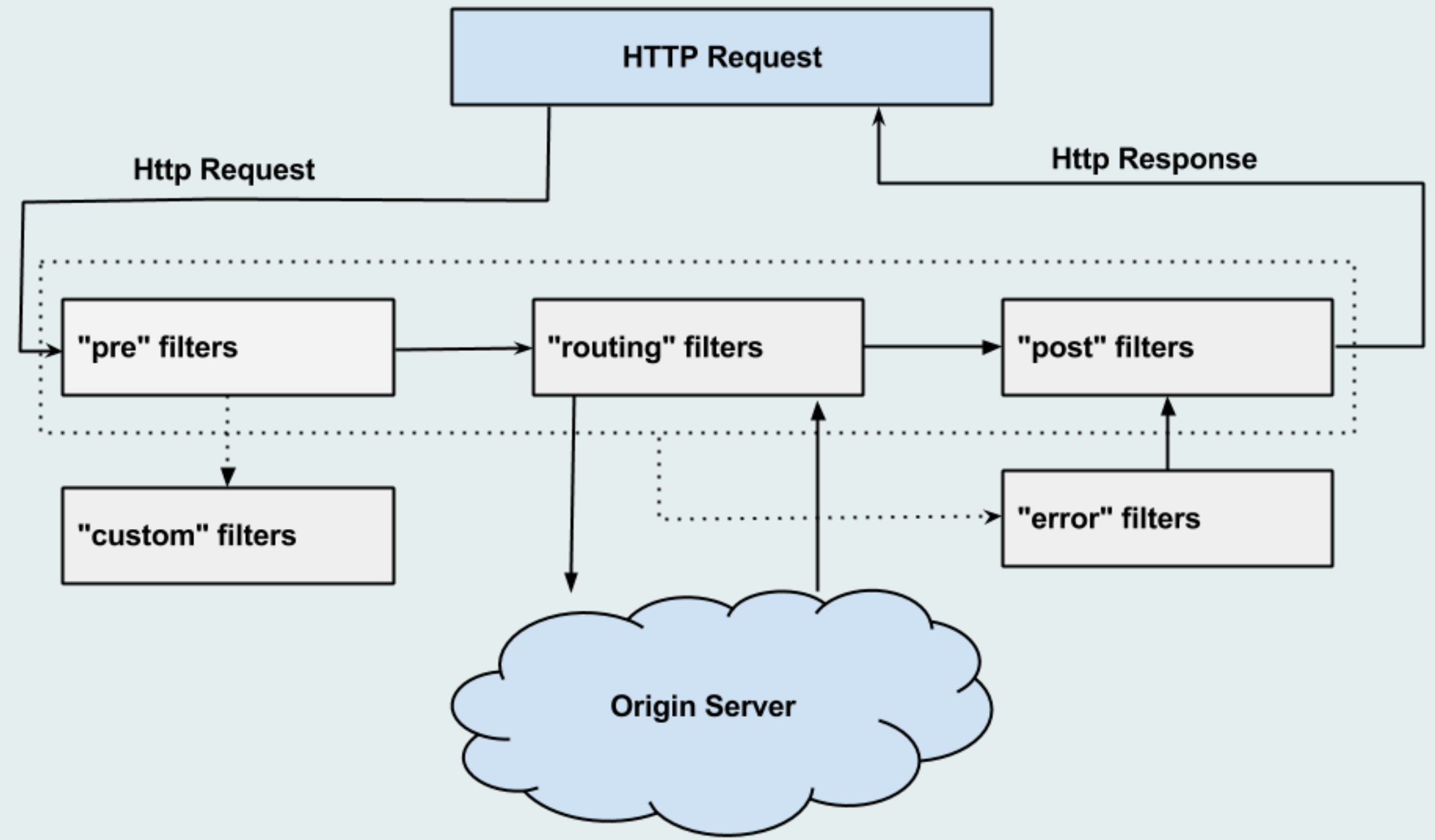
Spring Cloud Netflix-Hystrix

In a distributed microservices architecture multiple services can be dependent on each other. To ensure fault tolerance of microservices Hystrix introduced the circuit-breaker pattern.

For the Circuit Breaker to work, Hystrix will scan @Component or @Service annotated classes for @HystrixCommand annotated methods, implement a proxy for it and monitor its calls.

Spring Cloud Zuul

Zuul Server is an API Gateway application. It handles all the requests and performs the dynamic routing of microservice applications. It works as a front door for all the requests.



Declarative REST services: Feign

Feign is a declarative web service client. It makes writing web service clients easier.

To include Feign in your project use the starter with group `org.springframework.cloud` and artifact id `spring-cloud-starter-openfeign`.

```
@SpringBootApplication
@EnableFeignClients
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

}
```

Feign Client

To use Feign create an interface and annotate it. In the `@FeignClient` annotation the String value is the client name, which is used to create a Ribbon load balancer. You can also specify a URL using the `url` attribute (absolute value or just a hostname). The Ribbon client above will want to discover the physical addresses for the service. If your application is a Eureka client then it will resolve the service in the Eureka service registry.

```
@FeignClient("stores")
public interface StoreClient {
    @GetMapping(value= "/stores")
    List<Store> getStores();

    @GetMapping( value = "/stores/{storeid}", consumes = "application/json")
    Store update(@PathVariable("storeid") Long storeid, Store store);
}
```

Microservice Communication

Now that we have different working microservices, Let's understand how they exchange data



**But do we always know
the port numbers?**

Of course knowing the IP and port of each microservice we can make calls from one microservice to another.

Synchronous vs Asynchronous Communication

Synchronous Communication involves making HTTP calls, where in order for the information exchange to complete, the counterparty must process the request and get back to the requester with a corresponding response.

Introduces coupling into the system.

Time costs, request needs to be processed and sent back to the initial service , so that it can continue it's work

The failure of individual services might affect the whole application

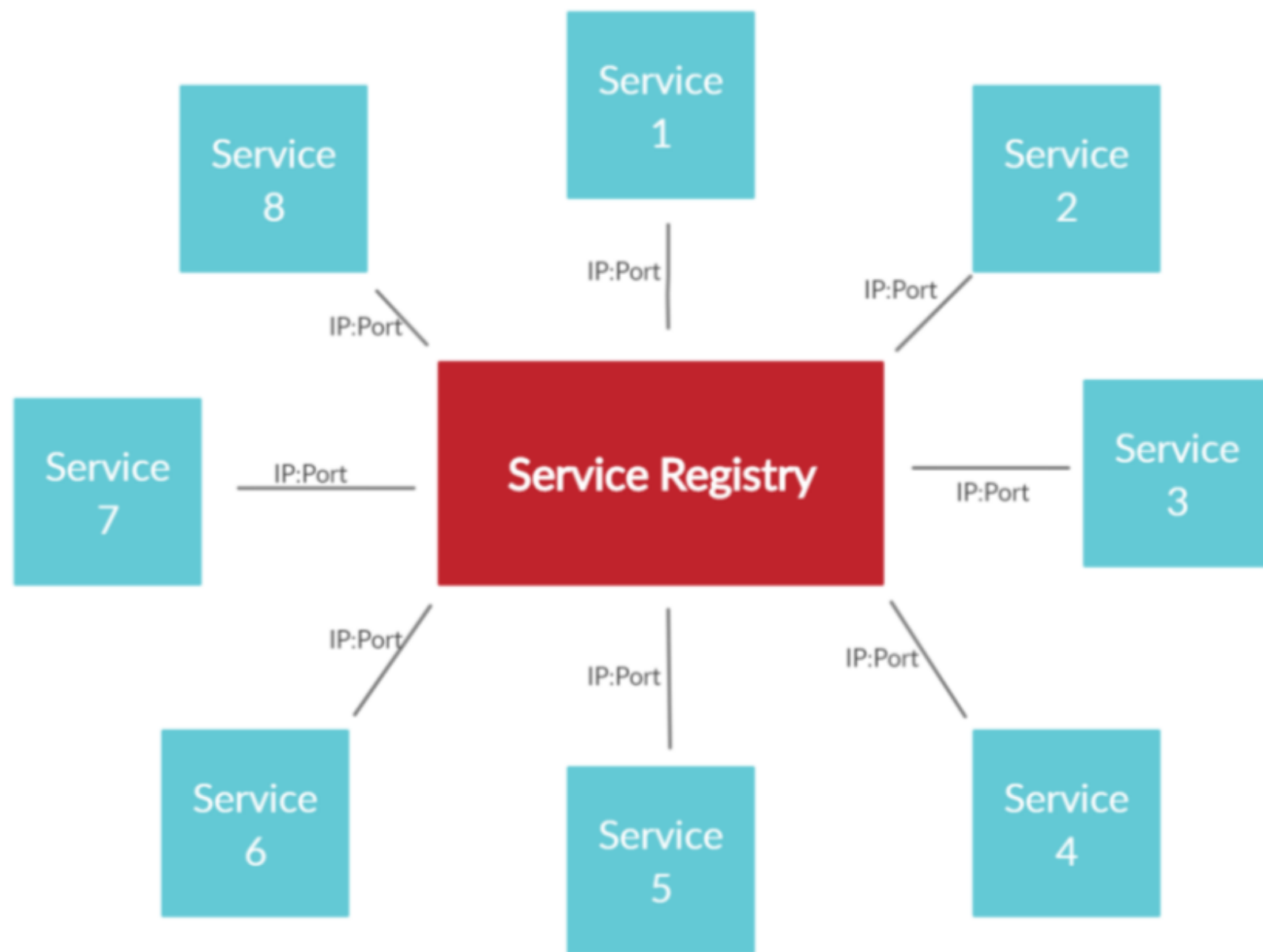
Asynchronous communication involves a message broker that plays the role of mediator between two microservices.

Decouples the services

Services communicate through message broker and not directly with each other, thus the response will be much quicker

The failure of individual services won't affect the whole application

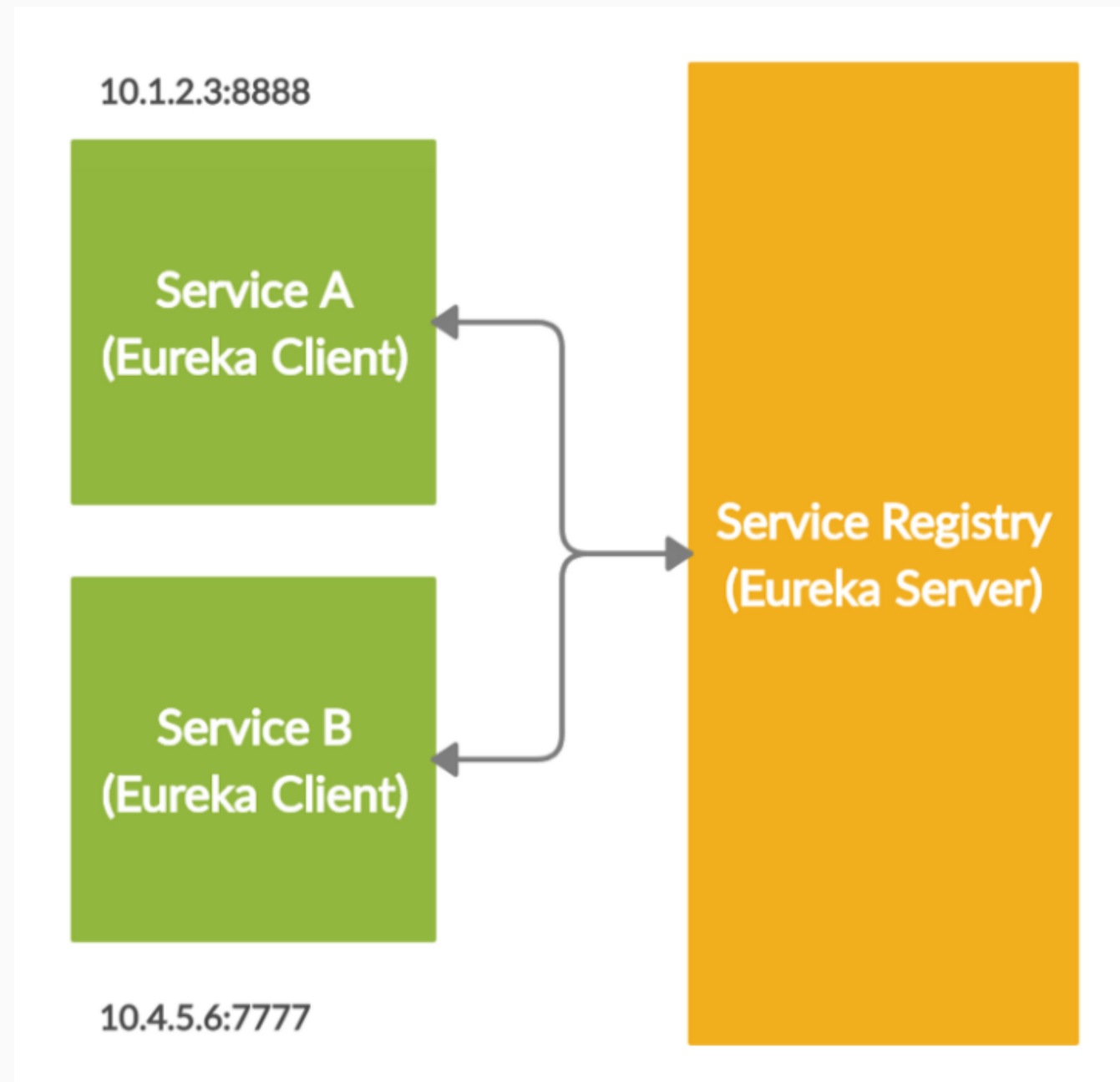
Service Discovery



Service discovery is the process of one service dynamically discovering the network location (IP address and port) of another service to communicate with it.

Service discovery mechanism uses a central registry to maintain the network locations of all the microservices. If for some reason the IP address and the port number of a particular microservice changes, new values will be immediately re-registered in the registry.

Spring Cloud Eureka



Eureka is a REST based service which is primarily used for acquiring information about services that you would want to communicate with. This REST service is also known as Eureka Server. The Services that register in Eureka Server to obtain information about each other are called Eureka Clients.

Eureka Server



Eureka Server is in itself a microservice to which all other microservices register.

The Eureka server does not have a back end store, but the service instances in the registry all have to send heartbeats to keep their registrations up to date (so this can be done in memory). Clients also have an in-memory cache of Eureka registrations (so they do not have to go to the registry for every request to a service).

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  <version>2.2.5.RELEASE</version>
</dependency>
```

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```


Eureka Server



By default, every Eureka server is also a Eureka client and requires (at least one) service URL to locate a peer. If you do not provide it, the service runs and works, but it fills your logs with a lot of noise about not being able to register with the peer.

To switch off the client-side behavior of Eureka Server we must set `registerWithEureka` and `fetchRegistry` to false

```
server:
  port: 8761

eureka:
  instance:
    hostname: localhost
  client:
    registerWithEureka: false
    fetchRegistry: false
```

Eureka Server



 **spring** Eureka

[HOME](#) [LAST 1000 SINCE STARTUP](#)

System Status

Environment	test
Data center	default

Current time	2021-02-12T21:57:25 +0400
Uptime	00:00
Lease expiration enabled	false
Renews threshold	1
Renews (last min)	0

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
No instances available			

General Info

Name	Value
total-avail-memory	296mb

Eureka Client

@EnableEurekaClient makes the app into both a Eureka "instance" (i.e. it registers itself) and a "client" (i.e. it can query the registry to locate other services).

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>  
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>  
  <version>2.2.5.RELEASE</version>  
</dependency>
```

```
@SpringBootApplication  
@EnableEurekaClient  
public class MovieCatalogServiceApplication {  
  public static void main(String[] args) {  
    SpringApplication.run(MovieCatalogServiceApplication.class, args);  
  }  
}
```

Eureka Client

Once the Eureka Client application is up and running we can check the Eureka Server page and see that it was added under Applications

server:
 port: 8090
spring:
 application:
 name: movie-catalog-service

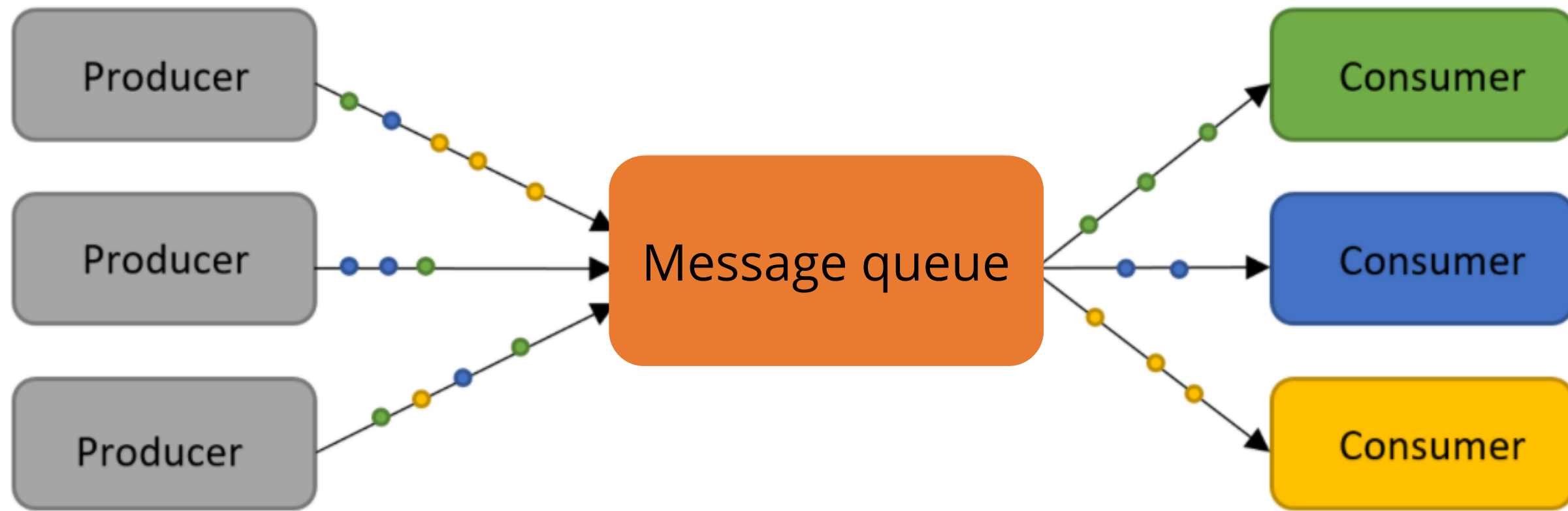
DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
MOVIE-CATALOG-SERVICE	n/a (1)	(1)	UP (1) - host.docker.internal:movie-catalog-service:8090

Message Broker



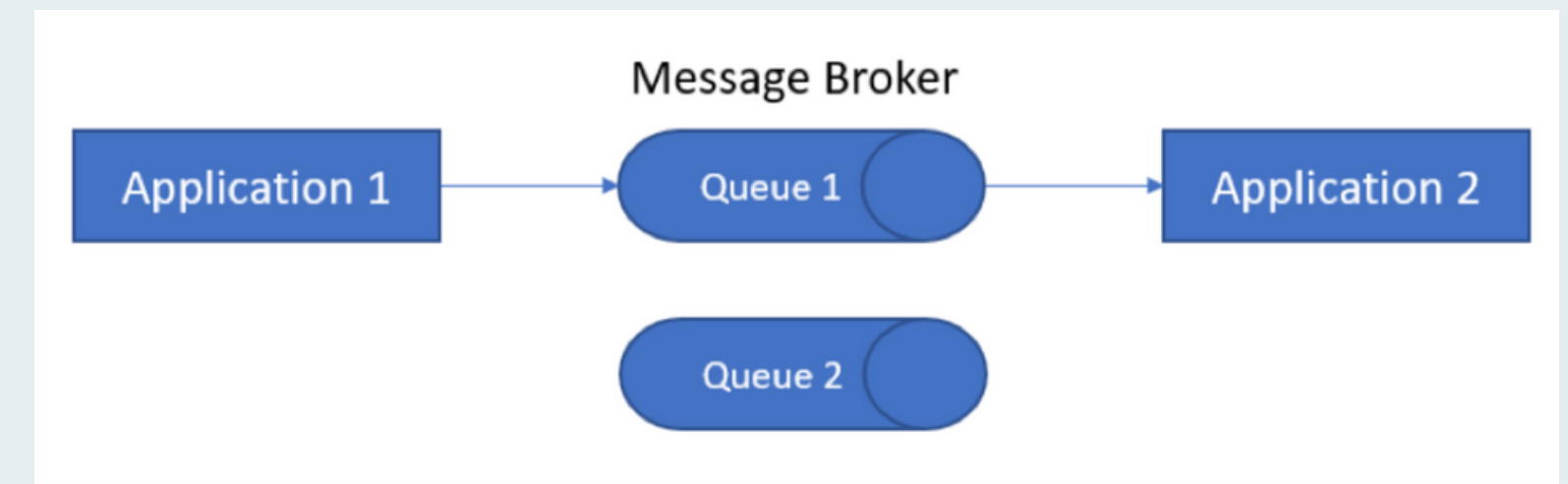
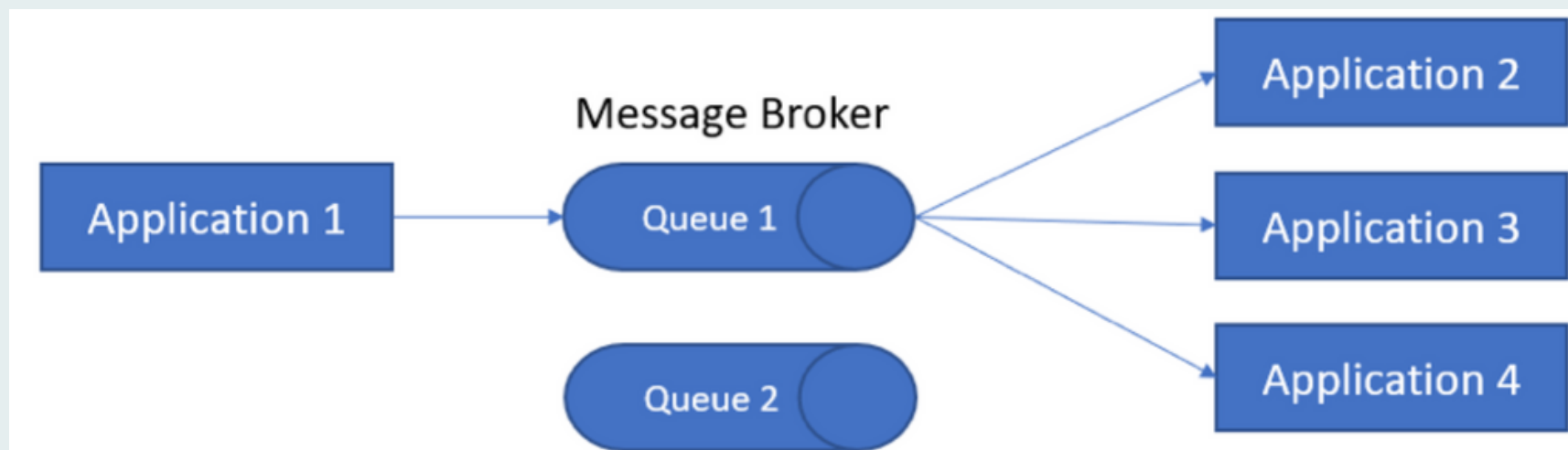
A message broker is software that enables applications, systems, and services to communicate with each other and exchange information.

In order to provide reliable message storage and guaranteed delivery, message brokers often rely on a substructure or component called a message queue.

Choosing a Message broker

Take into consideration

- Broker Scale (how many messages per second)
- Persistency(ability to recover messages)
- Consumer Capability(supports one-to-one or one-to many)



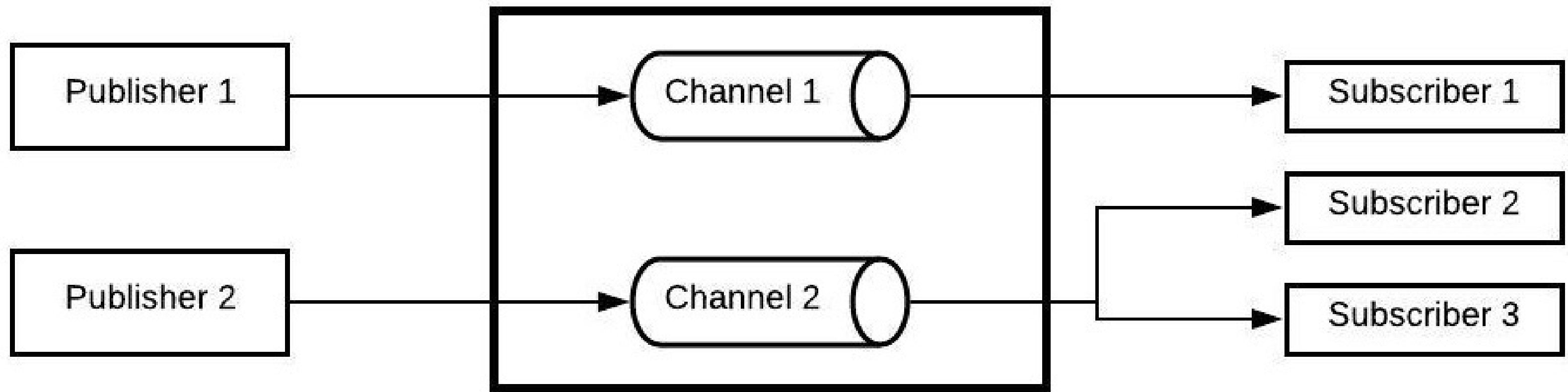
Kafka vs RabbitMQ vs Redis

- up to a million messages per second
- persistent Data
- only one-to-many

- around fifty thousand messages per second
- persistent and transient messages are supported
- one-to-one and one-to-many consumers

- up to a million messages per second
- not persistent – it's an in-memory datastore
- one-to-one and one-to-many consumers

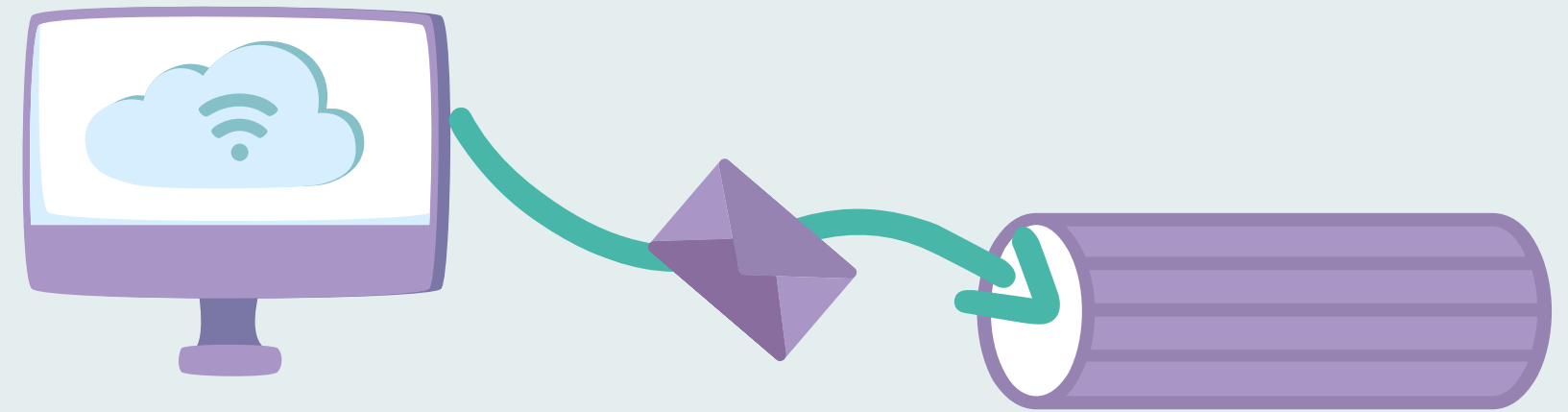
Redis Messaging System



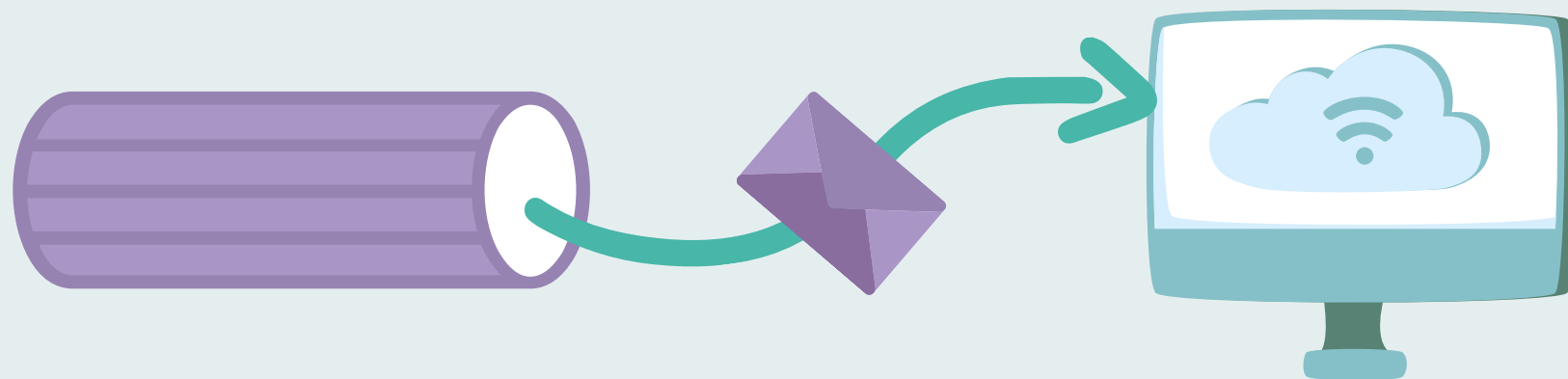
Redis Pub/Sub

Pub/Sub

Publishers are not programmed to send their messages to specific subscribers. Rather, published messages are characterized into channels, without knowledge of what (if any) subscribers there may be



Subscribers express interest in one or more channels, and only receive messages that are of interest, without knowledge of what (if any) publishers there are.

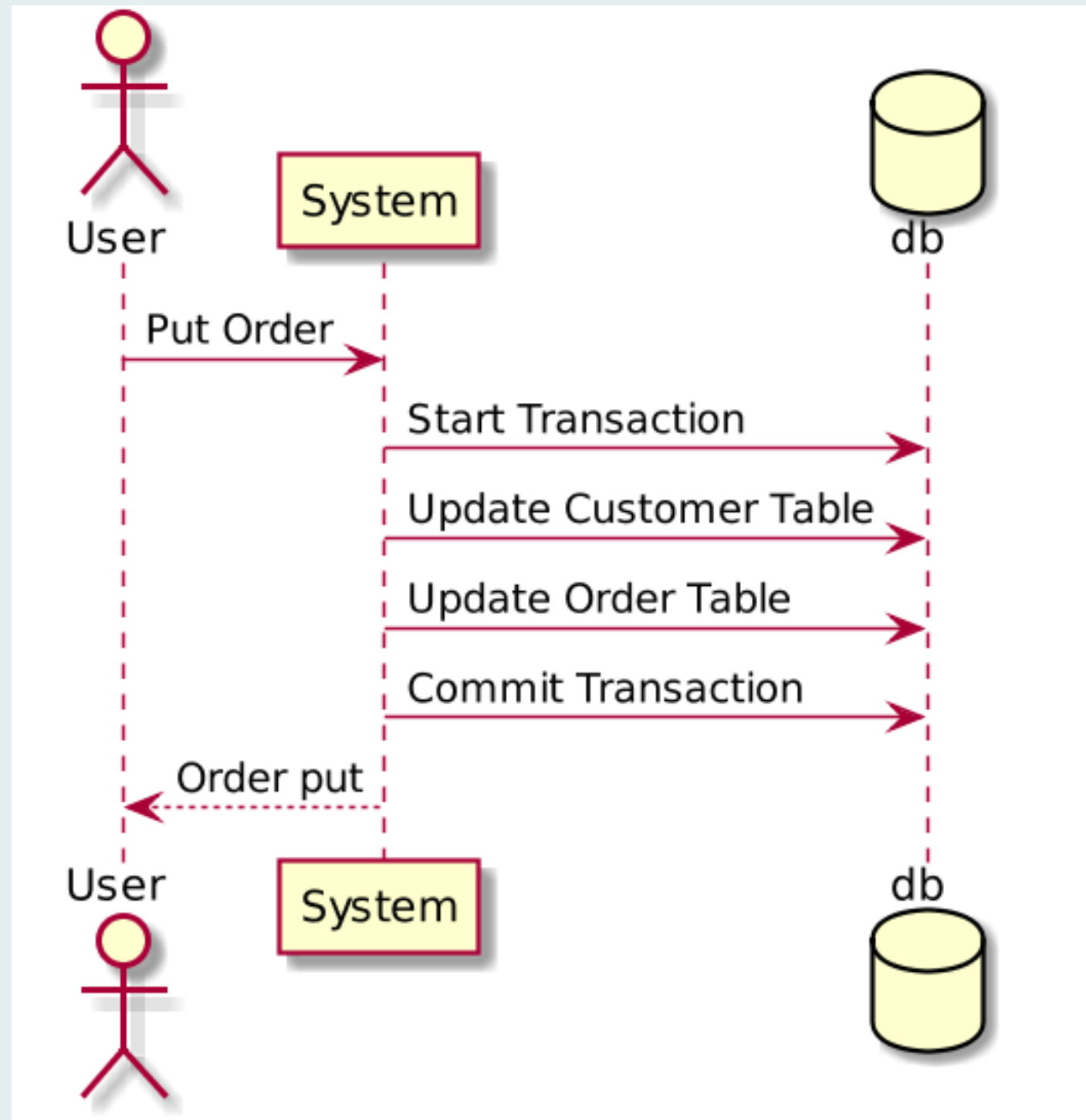


What is a distributed transaction?



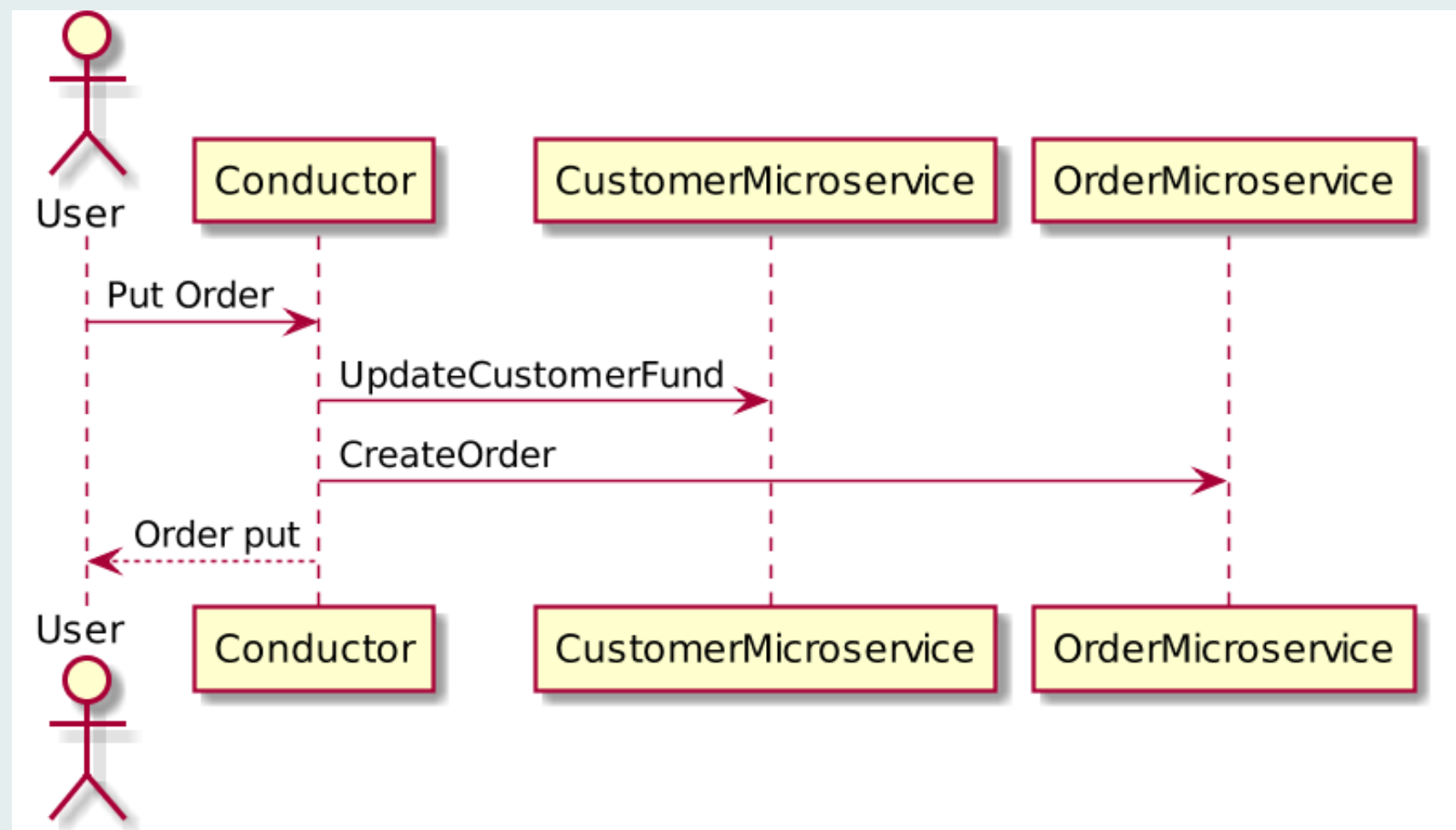
When a microservice architecture decomposes a monolithic system into self-encapsulated services, it can break transactions. This means a local transaction in the monolithic system is now distributed into multiple services that will be called in a sequence.

Transaction in monolithic system



In the customer order example, if a user sends a Put Order action to a monolithic system, the system will create a local database transaction that works over multiple database tables. If any step fails, the transaction can roll back. This is known as ACID (Atomicity, Consistency, Isolation, Durability), which is guaranteed by the database system.

Distributed transaction

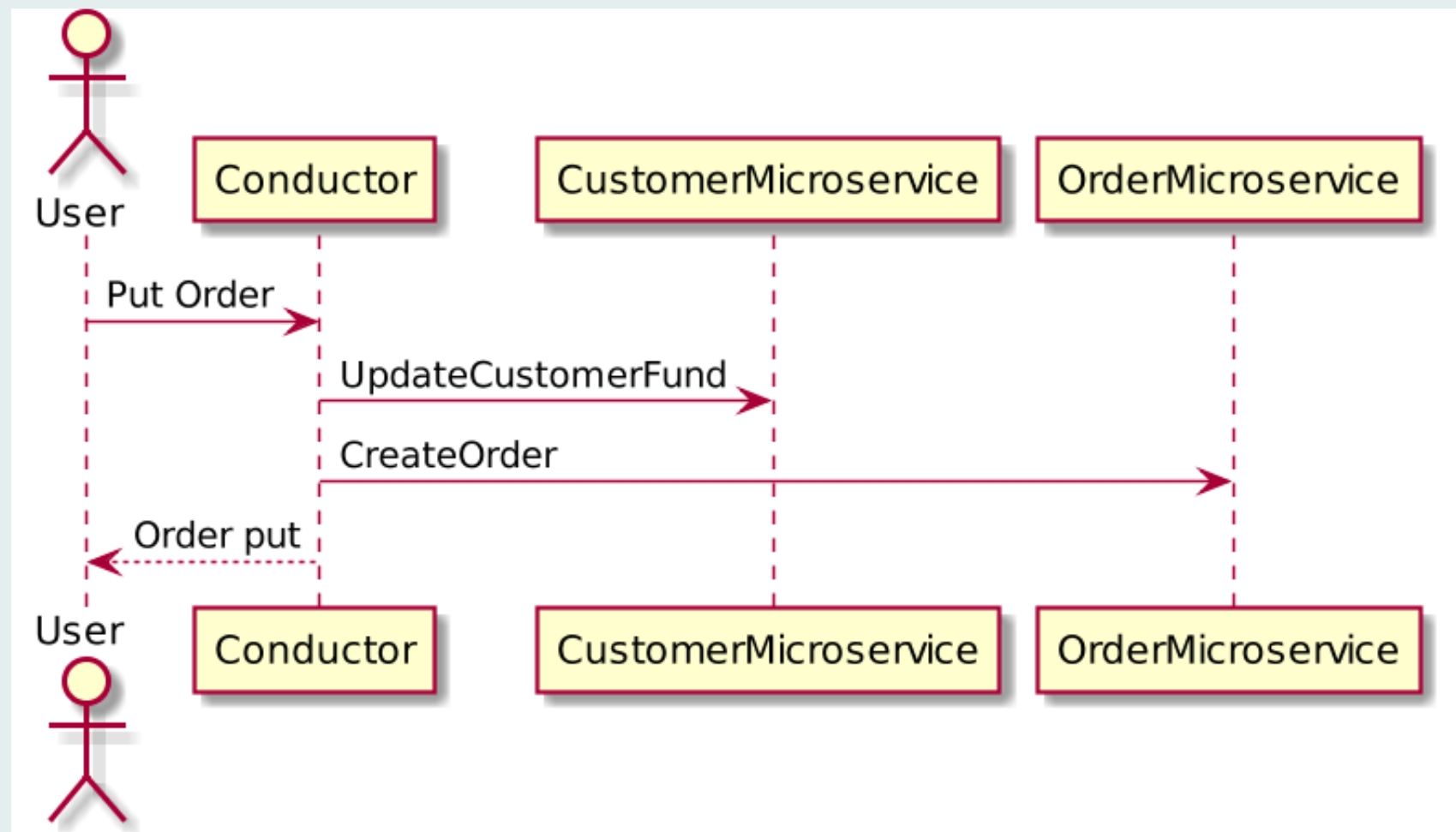


When we decompose this system, we created both the CustomerMicroservice and the OrderMicroservice, which have separate databases. Here is a customer order example with microservices:

When a Put Order request comes from the user, both microservices will be called to apply changes into their own database. Because the transaction is now across multiple databases, it is now considered a distributed transaction.

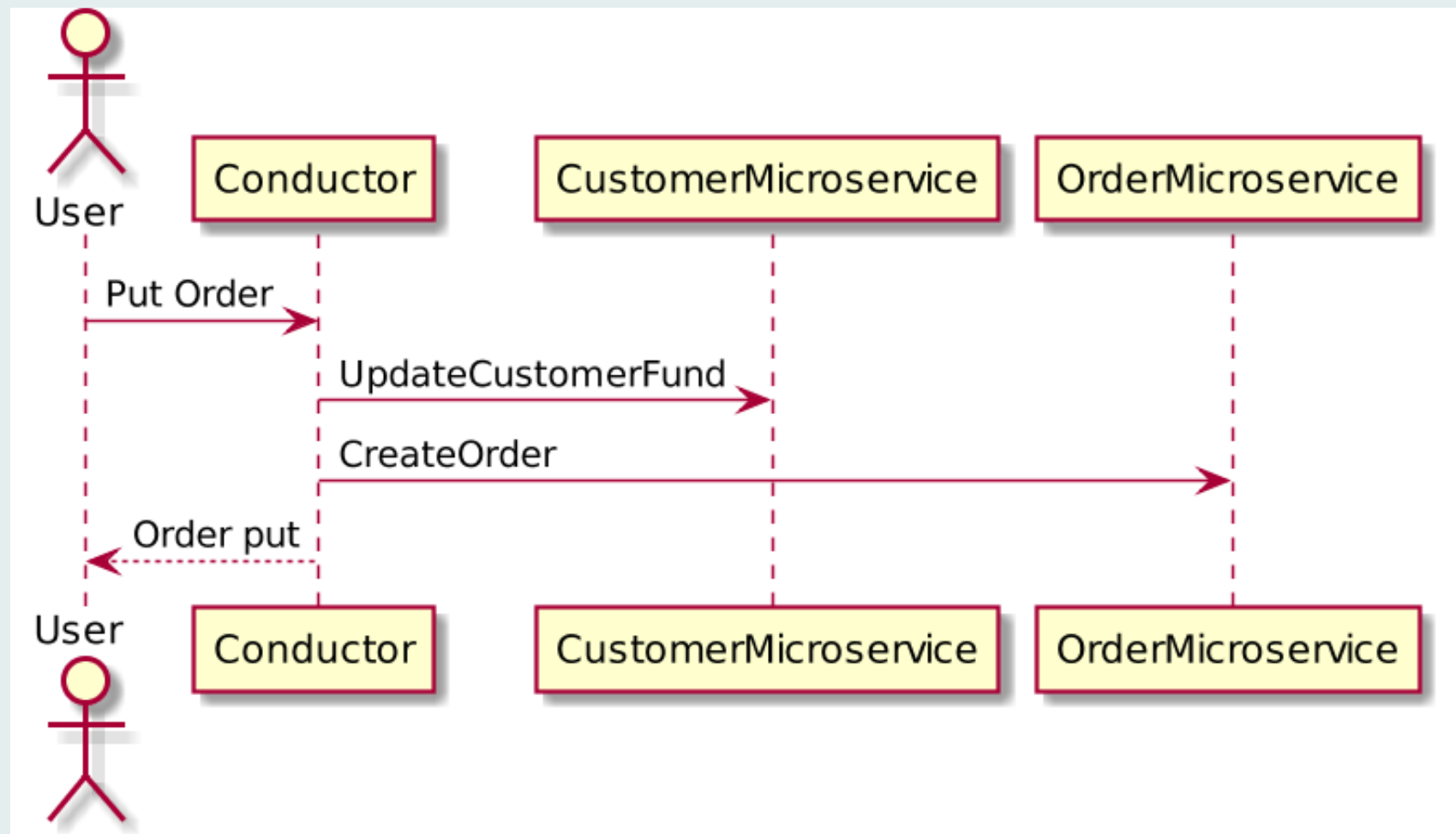
What is the problem?

How do we keep the transaction atomic?



In a database system, atomicity means that in a transaction either all steps complete or no steps complete. The microservice-based system does not have a global transaction coordinator by default. In the example, if the `CreateOrder` method fails, how do we roll back the changes we applied by the `CustomerMicroservice`?

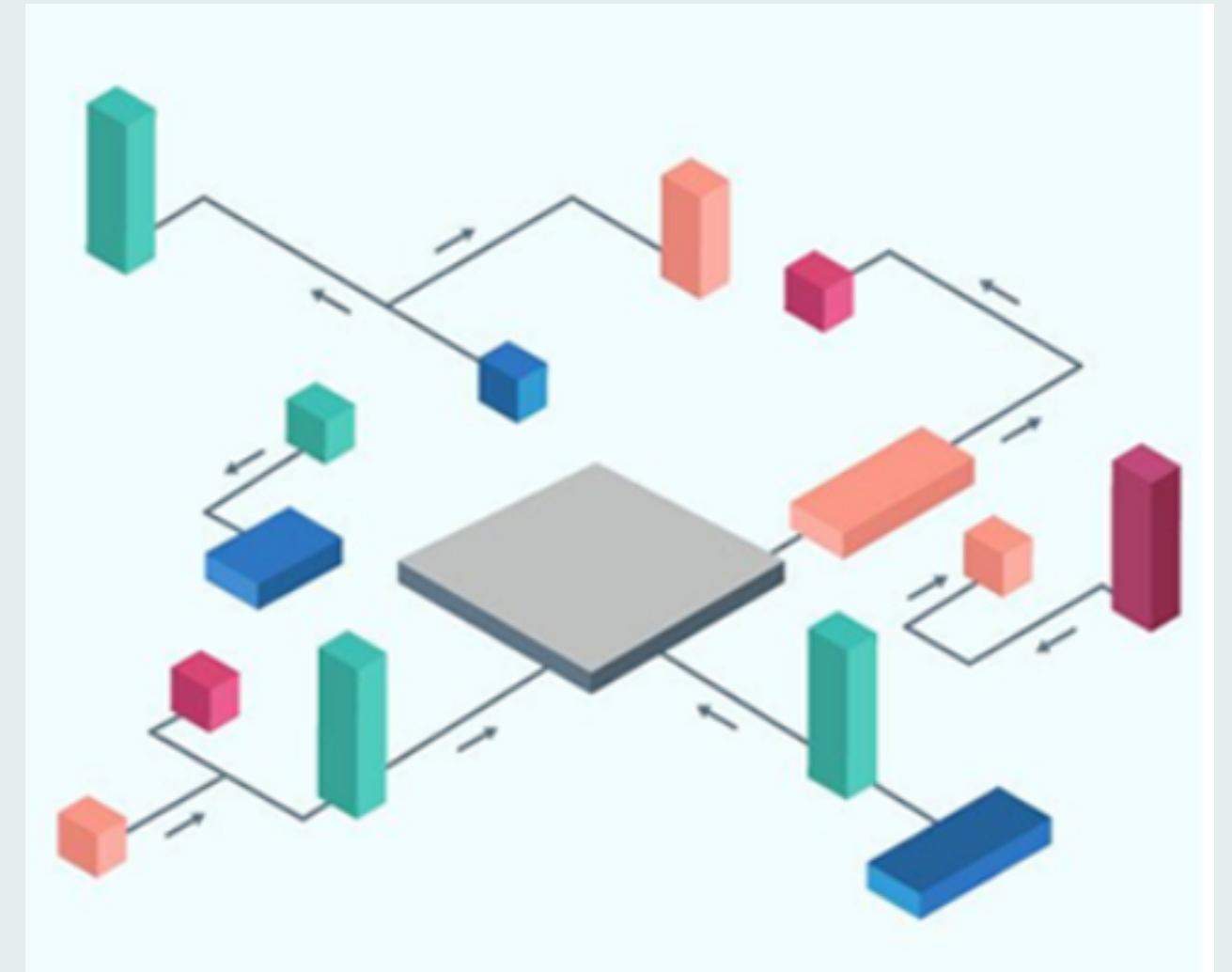
Do we isolate user actions for concurrent requests?



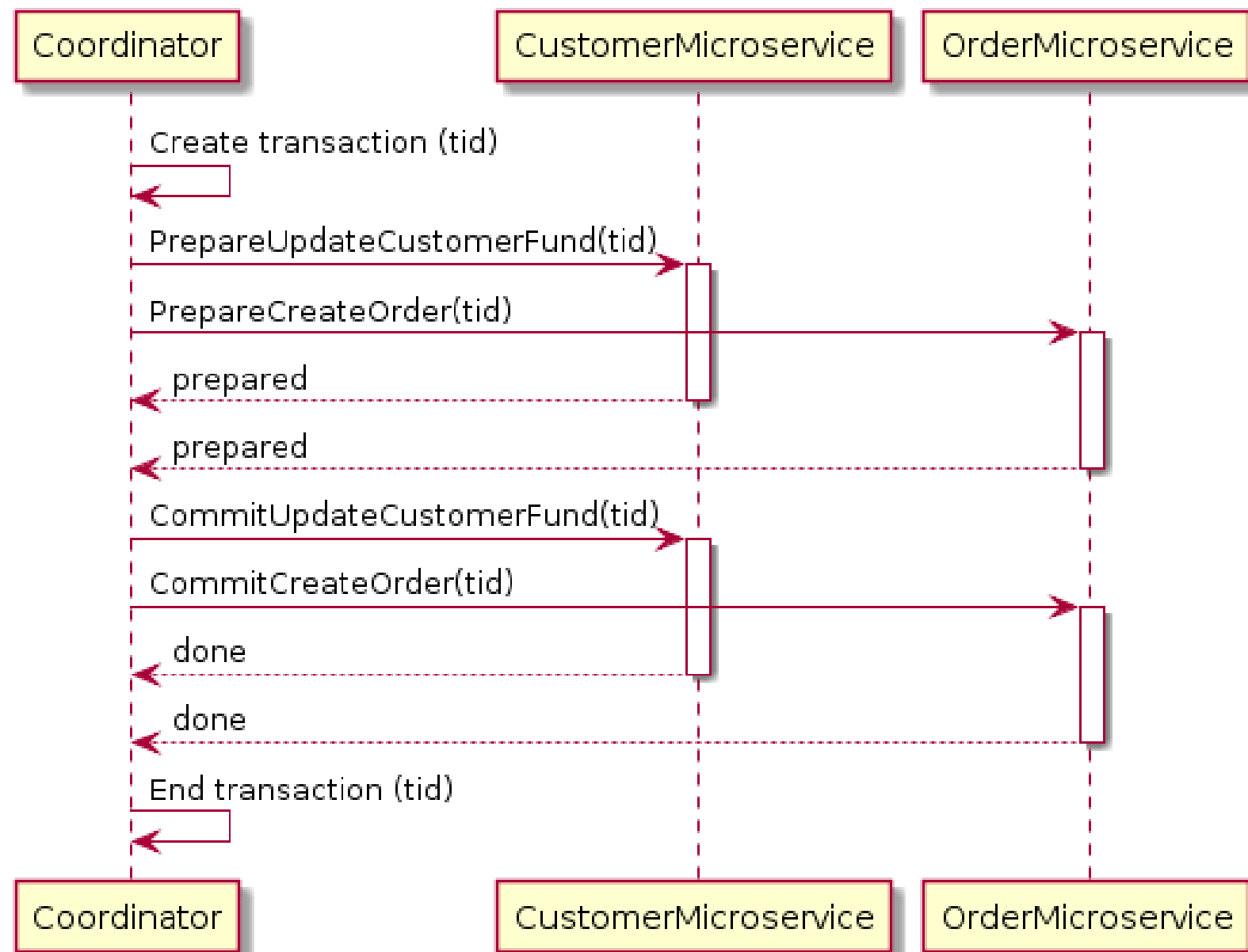
If an object is written by a transaction and at the same time (before the transaction ends), it is read by another request, should the object return old data or updated data? In the example, once `UpdateCustomerFund` succeeds but is still waiting for a response from `CreateOrder`, should requests for the current customer's fund return the updated amount or not?

Possible solutions

- 2pc (two-phase commit)
- Saga

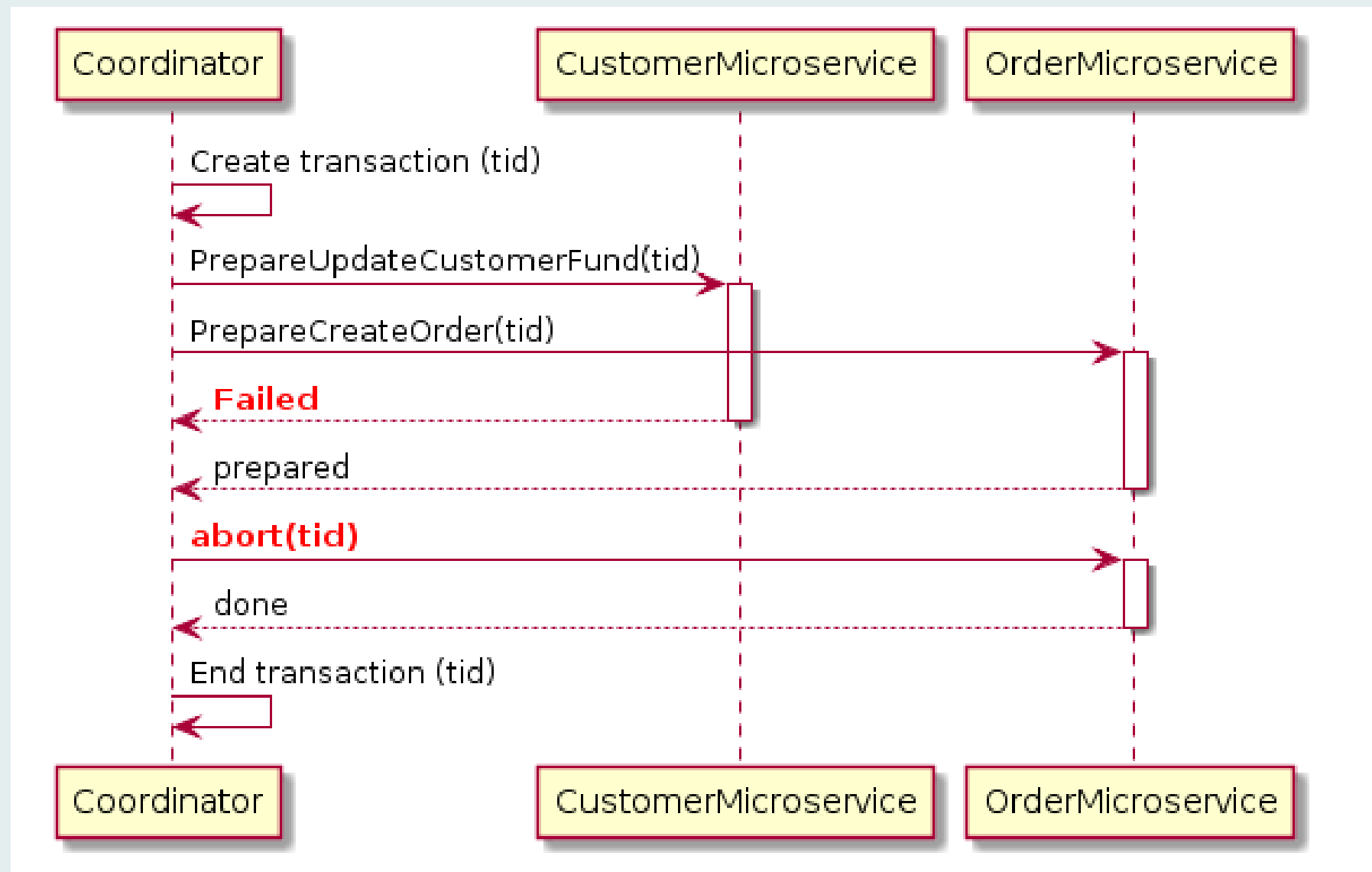


2pc (two-phase commit)



As its name hints, 2pc has two phases: A prepare phase and a commit phase. In the prepare phase, all microservices will be asked to prepare for some data change that could be done atomically. Once all microservices are prepared, the commit phase will ask all the microservices to make the actual changes. Normally, there needs to be a global coordinator to maintain the lifecycle of the transaction, and the coordinator will need to call the microservices in the prepare and commit phases.

2pc (two-phase commit)



If at any point a single microservice fails to prepare, the Coordinator will abort the transaction and begin the rollback process. Here is a diagram of a 2pc rollback for the customer order example: In the above example, the CustomerMicroservice failed to prepare for some reason, but the OrderMicroservice has replied that it is prepared to create the order. The Coordinator will request an abort on the OrderMicroservice with the transaction and the OrderMicroservice will then roll back any changes made and unlock the database objects.

Pros

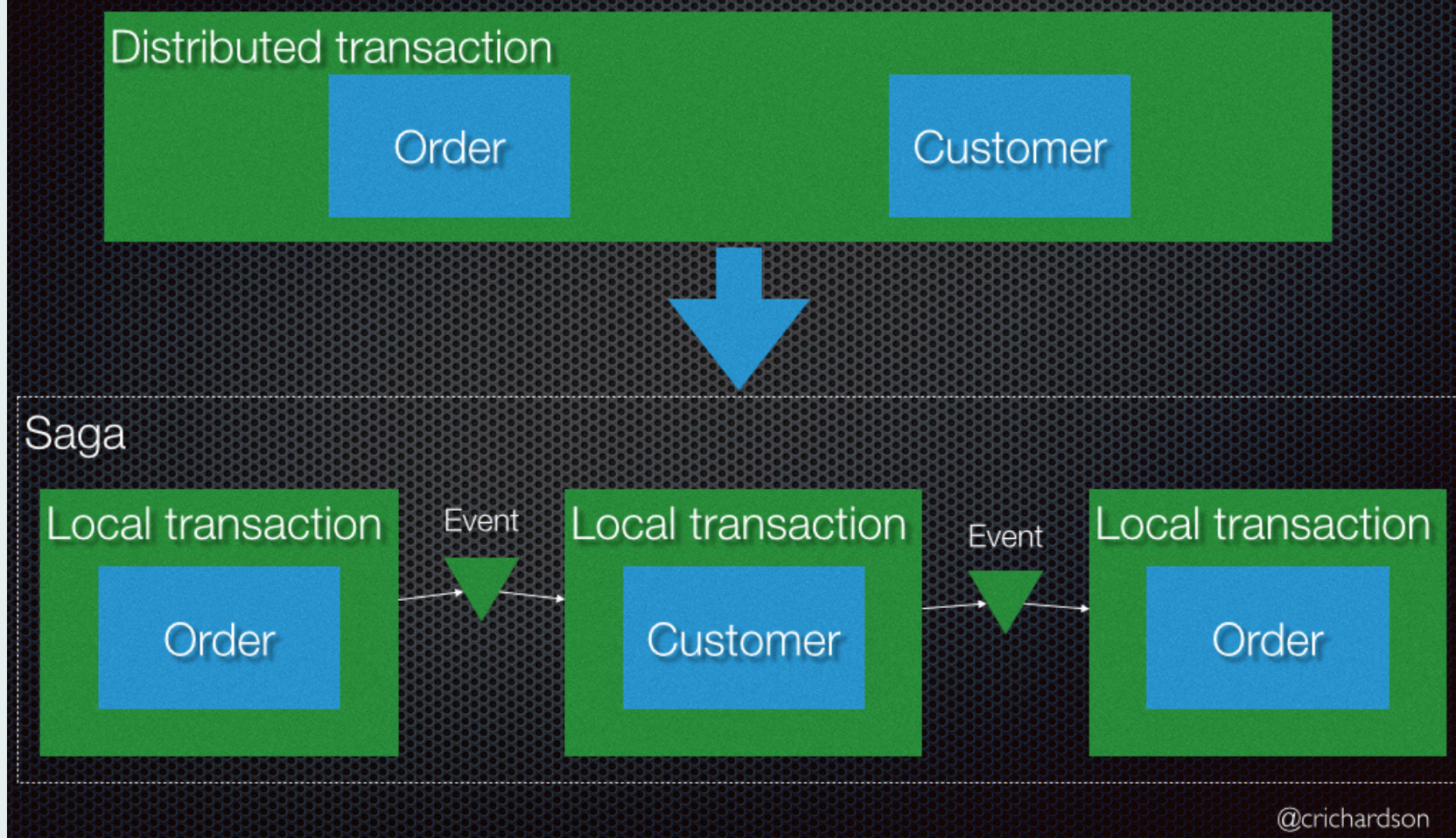
- Prepare and commit phases guarantee that the transaction is atomic.
- 2pc allows read-write isolation

Cons

2pc is synchronous (blocking) ●

Saga patterns

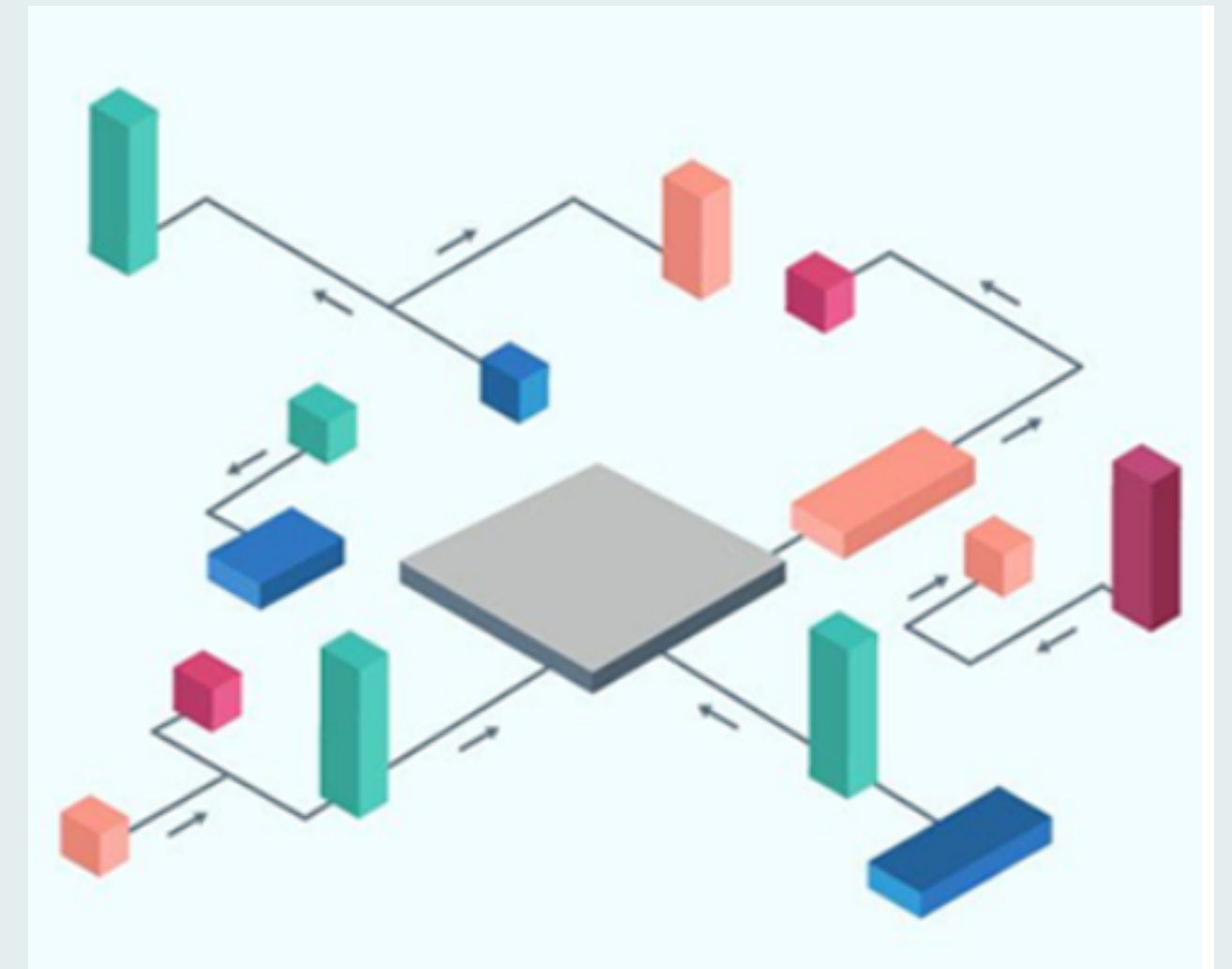
Using Sagas instead of 2PC



Sagas are mechanisms to maintain data consistency in a microservice architecture without having to use distributed transactions. You define a saga for each system command that needs to update data in multiple services. A saga is a sequence of local transactions. Each local transaction updates data within a single service using the familiar ACID transaction frameworks and libraries mentioned earlier. A service publishes a message when a local transaction completes. This message then triggers the next step in the saga. Not only does using messaging ensure the saga participants are loosely coupled, it also guarantees that a saga completes. That's because if the recipient of a message is temporarily unavailable, the message broker buffers the message until it can be delivered.

Saga patterns

- Choreography : each local transaction publishes domain events that trigger local transactions in other services.
- Orchestration : an orchestrator (object) tells the participants what local transactions to execute.



Choreography based sagas

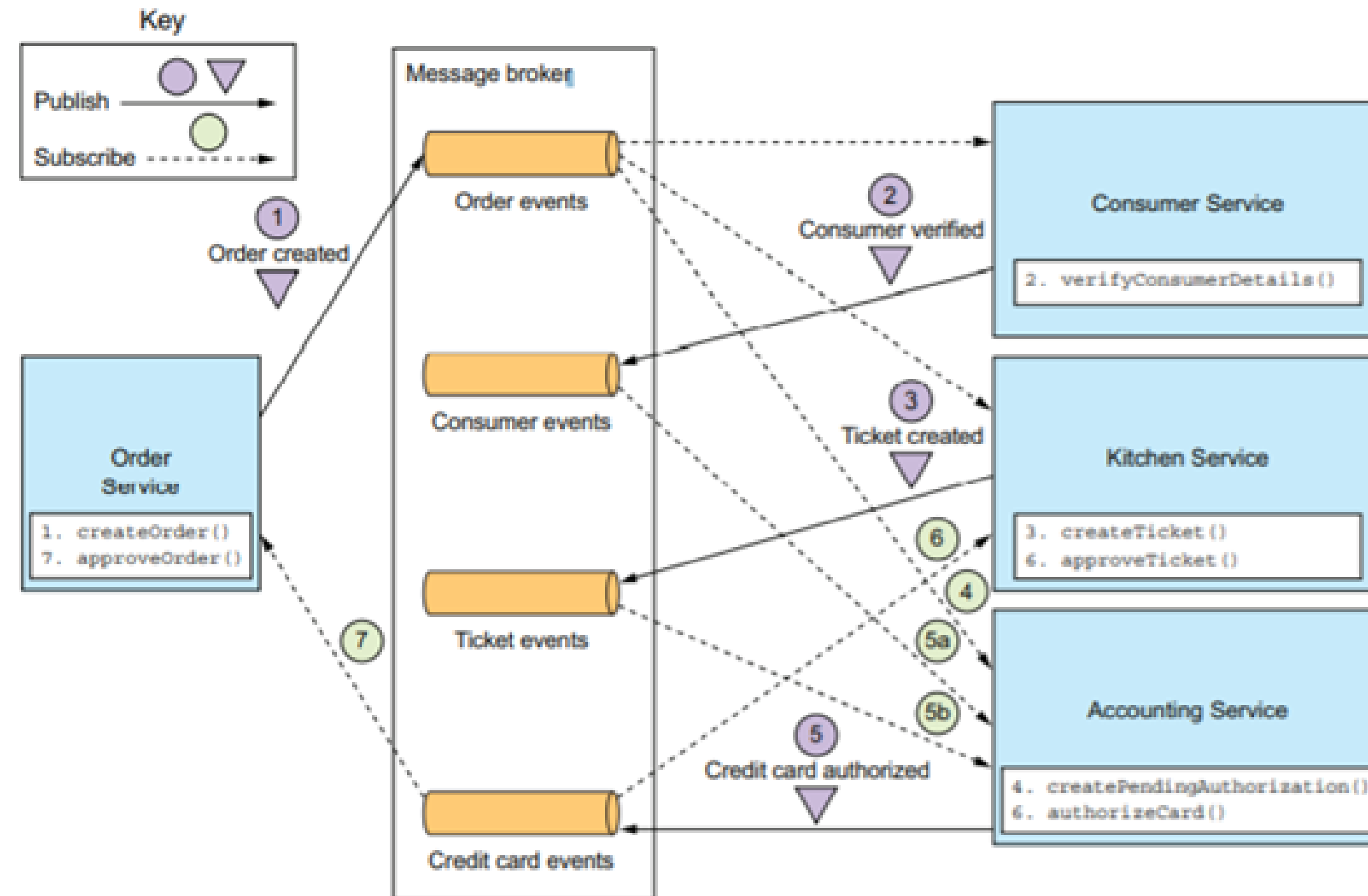


Figure 4.4 Implementing the Create Order Saga using choreography. The saga participants communicate by exchanging events.

Pros

- Loose coupling

Cons

- More difficult to understand for developers ●
- Cyclic dependencies between the services ●

Orchestration based sagas

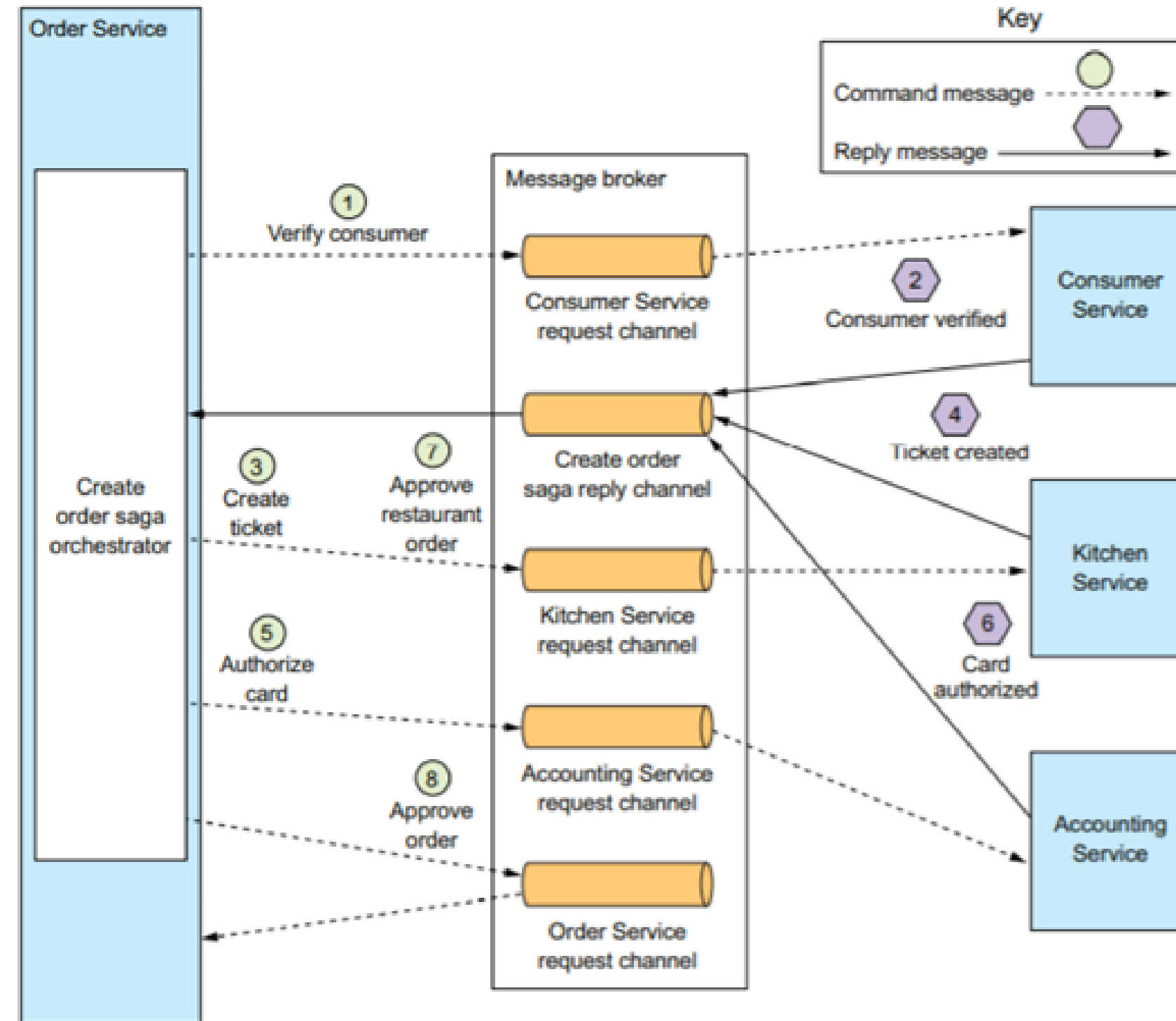


Figure 4.6 Implementing the Create Order Saga using orchestration. Order Service implements a saga orchestrator, which invokes the saga participants using asynchronous request/response.

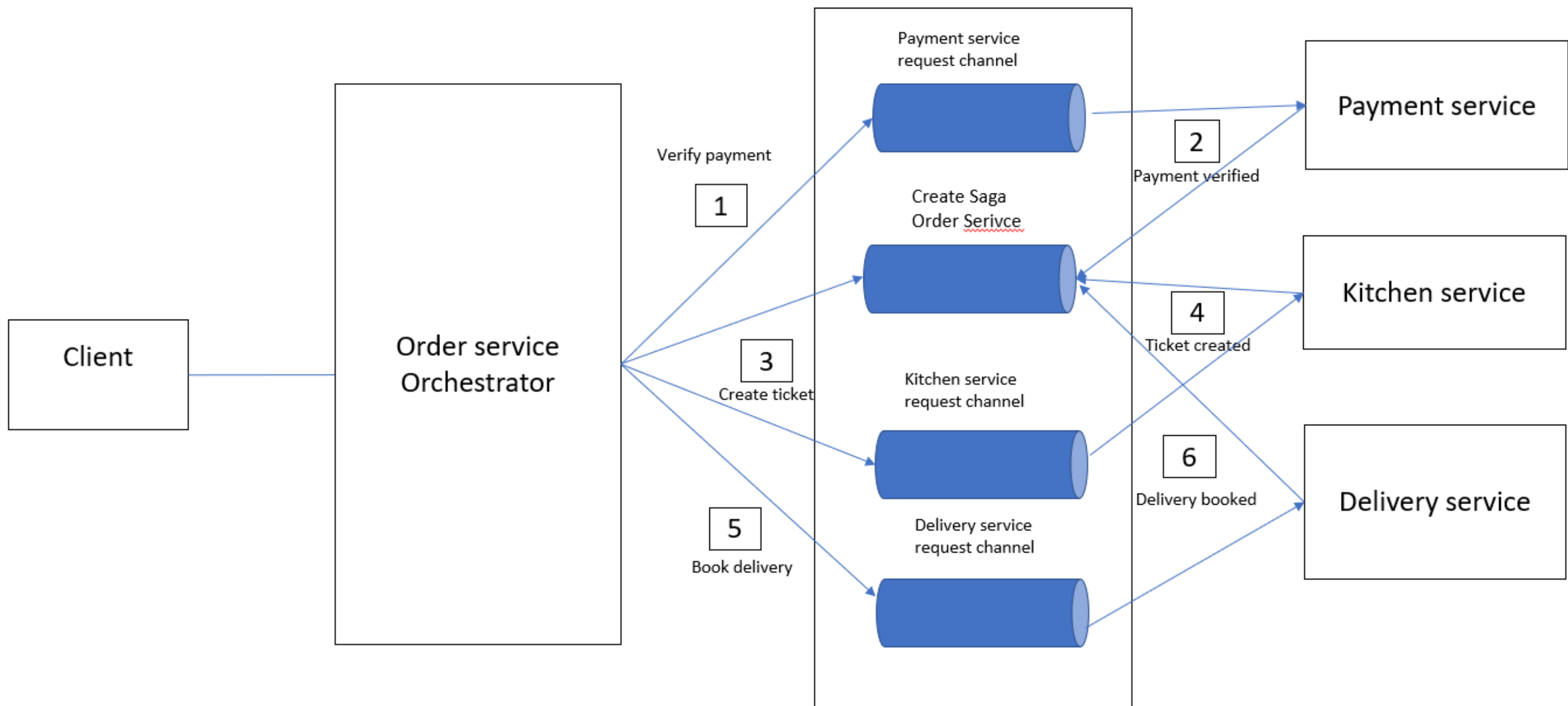
Pros

- No cyclic dependencies
- Less coupling
- Improves separation of concerns and simplifies the business logic

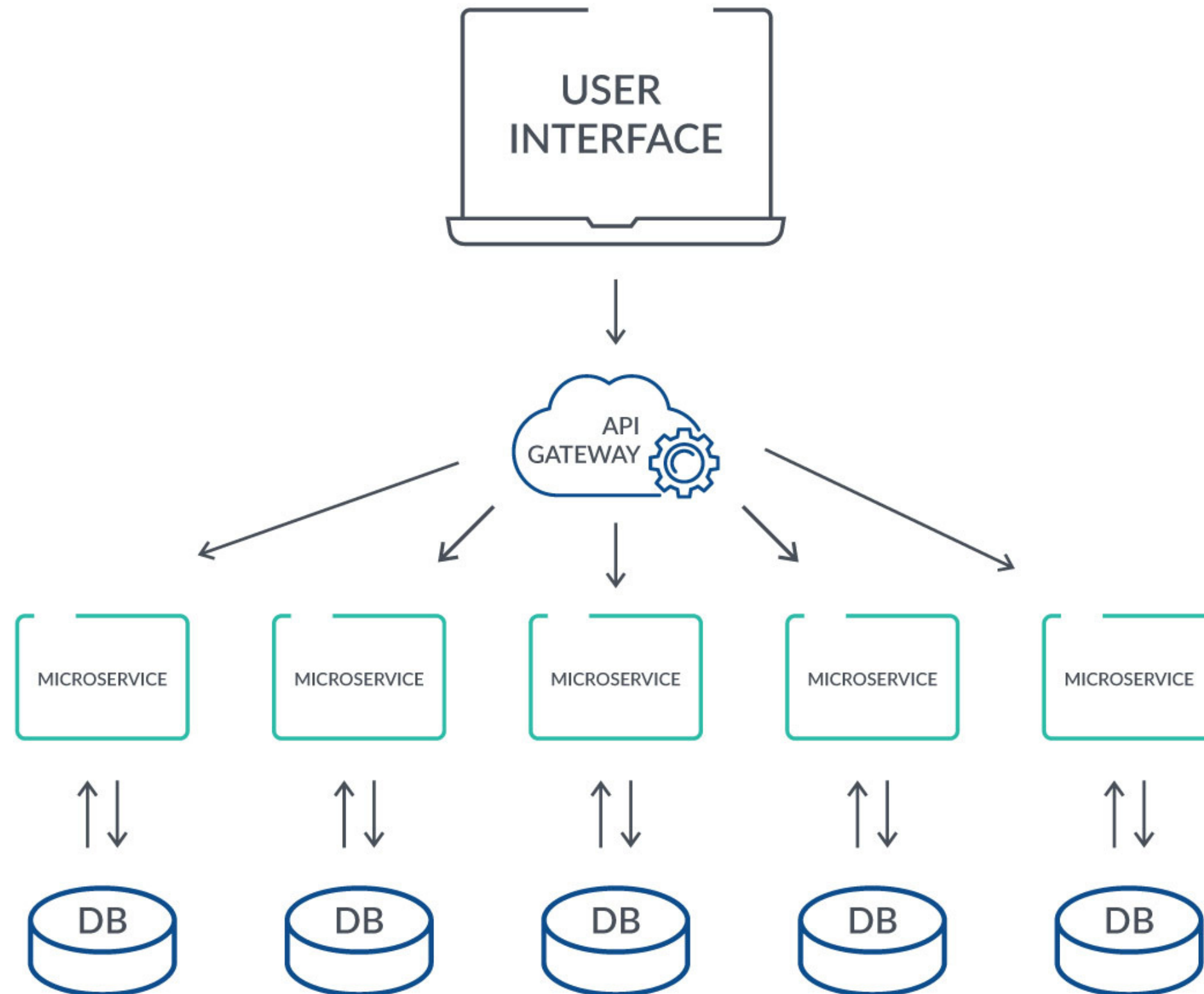
Cons

- The risk of centralizing too much business logic in the orchestrator

Saga patterns

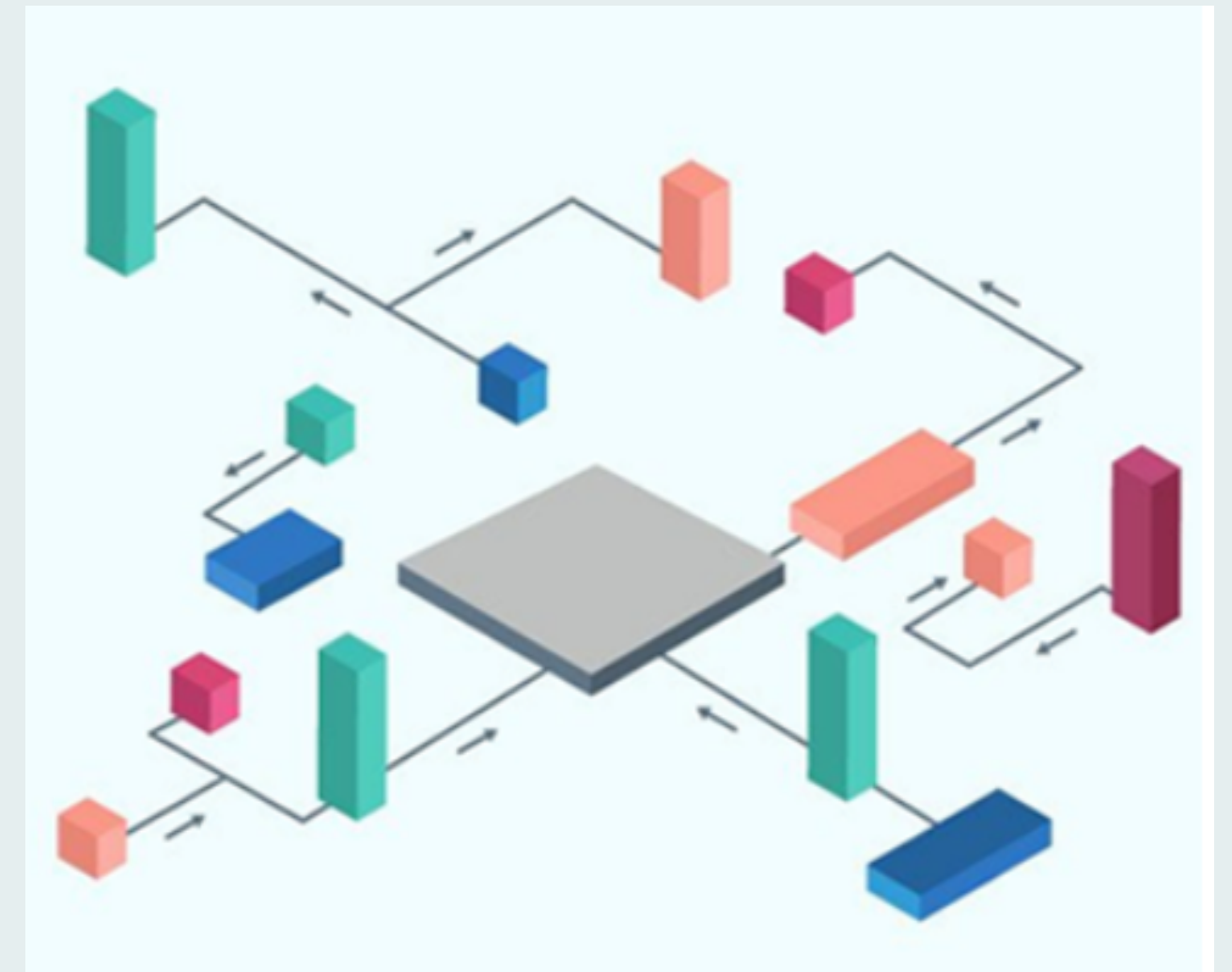


Implementing queries in a microservice architecture



Implementing queries in a microservice architecture

- API composition pattern
- Query responsibility segregation (CQRS) pattern



API composition pattern

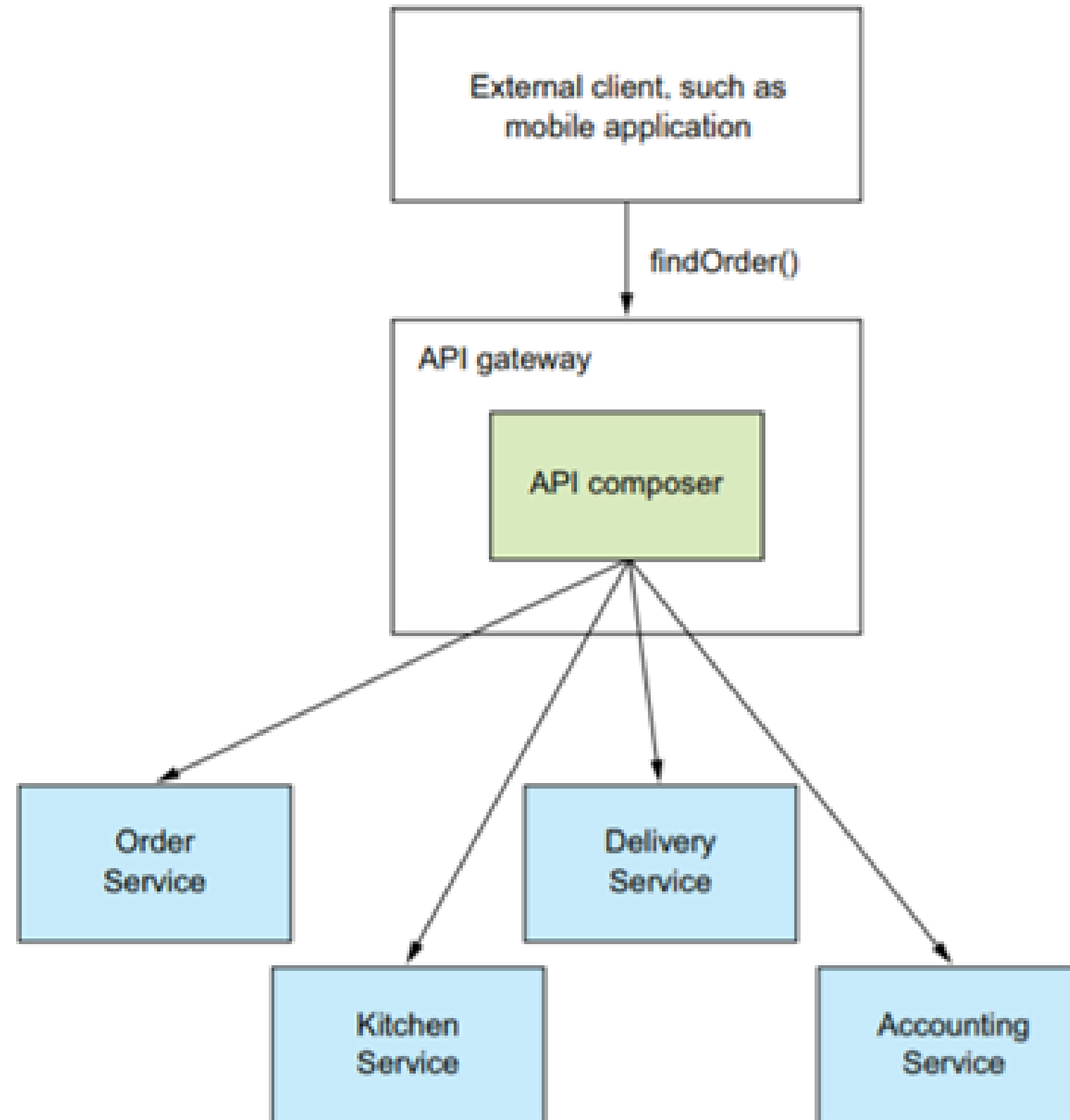


Figure 7.5 Implementing API composition in the API gateway. The API queries the provider services to retrieve the data, combines the results, and returns a response to the client.

Pros

- Simple and intuitive

Cons

- Increased overhead ●
- Risk of reduced availability ●
- Lack of transactional data consistency ●

CQRS pattern

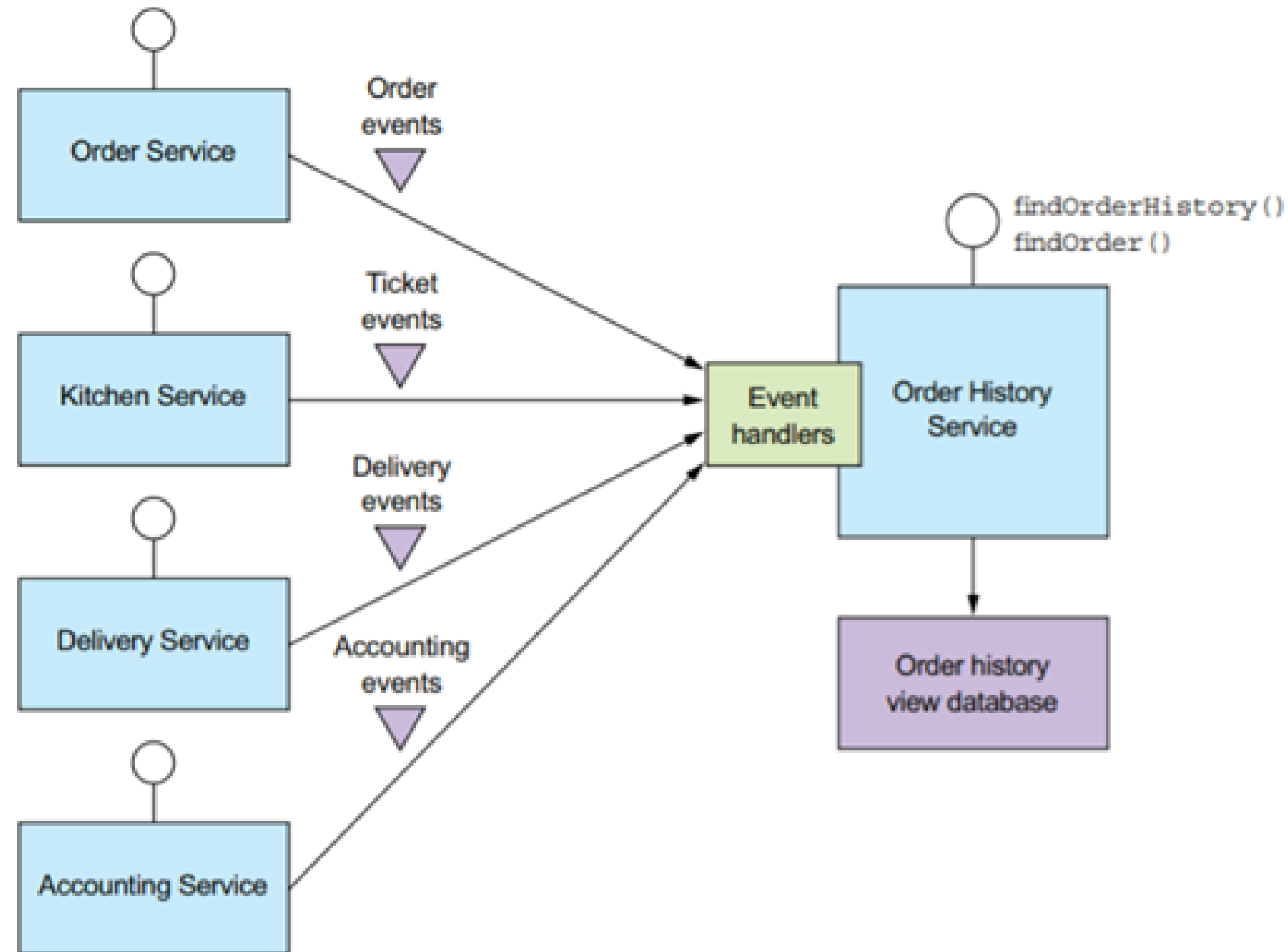


Figure 7.9 The design of Order History Service, which is a query-side service. It implements the `findOrderHistory()` query operation by querying a database, which it maintains by subscribing to events published by multiple other services.

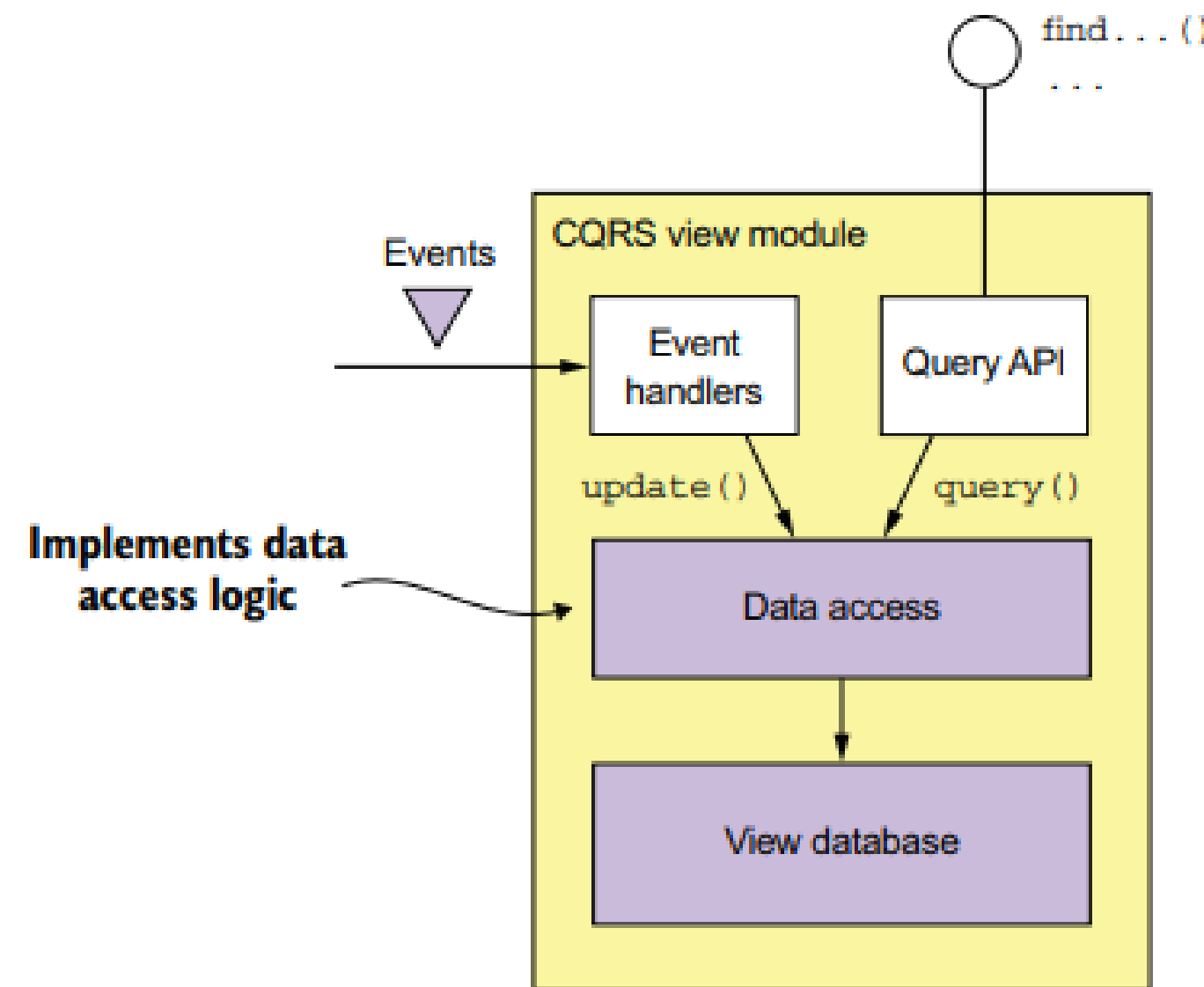


Figure 7.10 The design of a CQRS view module. Event handlers update the view database, which is queried by the Query API module.

Pros

- Enables the efficient implementation of queries in a microservice architecture

Cons

- More complex architecture •
- Dealing with the replication lag •



Thank

you